🛒 Retail Store Recommendation System

Using Association Rule Mining (Apriori Algorithm)

📌 Project Overview

This notebook builds a **Retail Product Recommendation System** using **Association Rule Mining** — specifically the **Apriori Algorithm**.

🤖 Learning Type

> ⚠️ **This is an UNSUPERVISED LEARNING approach.**
> There are no labels or target variables. The algorithm discovers hidden patterns and relationships between products purely from transaction co-occurrence data.

📊 Dataset

- **BASKET_ID** : Unique transaction/basket ID
- **COMMODITY_DESC** : List of products purchased together (stored as a string-formatted Python list)

🗺️ Roadmap

Step	Description
1	Import Libraries
2	Load & Explore Dataset
3	Data Preprocessing
4	Create Transaction Data
5	One-Hot Encoding
6	Apply Apriori Algorithm
7	Generate Association Rules
8	Build Recommendation Function
9	EDA Visualizations
10	Validation Metrics
11	Model Performance Visualizations
12	Interpretation
13	Save Outputs

📦 Step 1: Import Required Libraries

```
# ============================================================
# STEP 1: IMPORT REQUIRED LIBRARIES
# ============================================================

# Install mlxtend if not already installed (uncomment in Colab)
# !pip install mlxtend -q

# Core data manipulation
import pandas as pd
import numpy as np

# String parsing
import ast

# Visualization
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns

# Association Rule Mining
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# Warnings
import warnings
warnings.filterwarnings('ignore')

# Plot style
sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.dpi'] = 120
plt.rcParams['font.size'] = 11

print('✅ All libraries imported successfully!')
```

```
✅ All libraries imported successfully!
```

📂 Step 2: Load and Explore the Dataset

```
# ============================================================
# STEP 2: LOAD AND EXPLORE DATASET
# ============================================================

# ------------------------------------------------------------
# Load the CSV dataset
# Update the path if running locally.
# In Google Colab, upload the file and use the path below:
# ------------------------------------------------------------
df = pd.read_csv('transactions.csv')   # <-- Update path if needed

print('=' * 55)
print('         DATASET OVERVIEW')
```

```
print('=' * 55)
print(f'Shape  : {df.shape[0]} rows  x  {df.shape[1]} columns')
print(f'Columns: {list(df.columns)}')
print()
```

```
=======================================================
         DATASET OVERVIEW
=======================================================
Shape  : 7463 rows  x  2 columns
Columns: ['BASKET_ID', 'COMMODITY_DESC']
```

```
# Display first 5 rows
print('--- First 5 rows ---')
df.head()
```

```
--- First 5 rows ---
```

	BASKET_ID	COMMODITY_DESC
0	26984851472	['VEGETABLES - ALL OTHERS', 'TROPICAL FRUIT', ...
1	26984851516	['COOKIES/CONES', 'BROOMS AND MOPS', 'BAKED BR...
2	26984896261	['CRACKERS/MISC BKD FD', 'BREAKFAST SAUSAGE/SA...
3	26984905972	['SOUP', 'BAKED BREAD/BUNS/ROLLS']
4	26984945254	['ELECTRICAL SUPPPLIES', 'CANDY - PACKAGED', '...

Next steps: Generate code with df New interactive sheet

```
# Data types and non-null counts
print('--- Dataset Info ---')
df.info()
print()

# Missing values
print('--- Missing Values ---')
print(df.isnull().sum())
print()

# Sample COMMODITY_DESC values (raw string format)
print('--- Sample COMMODITY_DESC (raw string) ---')
for i, val in enumerate(df['COMMODITY_DESC'].head(5)):
    print(f'  Row {i}: {val}')
```

```
--- Dataset Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7463 entries, 0 to 7462
Data columns (total 2 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   BASKET_ID       7463 non-null   int64
 1   COMMODITY_DESC  7462 non-null   object
dtypes: int64(1), object(1)
memory usage: 116.7+ KB
```

```
--- Missing Values ---
BASKET_ID         0
COMMODITY_DESC    1
dtype: int64

--- Sample COMMODITY_DESC (raw string) ---
  Row 0: ['VEGETABLES - ALL OTHERS', 'TROPICAL FRUIT', 'ONIONS', 'ORGANICS FR
  Row 1: ['COOKIES/CONES', 'BROOMS AND MOPS', 'BAKED BREAD/BUNS/ROLLS', 'PNT
  Row 2: ['CRACKERS/MISC BKD FD', 'BREAKFAST SAUSAGE/SANDWICHES', 'EGGS', 'BE
  Row 3: ['SOUP', 'BAKED BREAD/BUNS/ROLLS']
  Row 4: ['ELECTRICAL SUPPPLIES', 'CANDY - PACKAGED', 'CANDY - CHECKLANE']
```

```
# ---------------------------------------------------------
# DATASET STRUCTURE EXPLANATION
# ---------------------------------------------------------
# Each row = 1 shopping basket / transaction
# BASKET_ID  : Unique numeric ID for the transaction
# COMMODITY_DESC : A Python-list-like string containing
#                  all product categories bought together
#                  Example: "['VEGETABLES', 'EGGS', 'MILK']"
# Goal: Find which products are frequently bought together
# and build a recommendation system using those patterns.
print('📖 Dataset structure understood.')
print(f'   Total transactions : {df.shape[0]}')
print(f'   COMMODITY_DESC type: {type(df["COMMODITY_DESC"].iloc[0])}')
```

```
📖 Dataset structure understood.
   Total transactions : 7463
   COMMODITY_DESC type: <class 'str'>
```

🧹 Step 3: Data Preprocessing

```
# ============================================================
# STEP 3: DATA PREPROCESSING
# ============================================================

def parse_commodity(raw_str):
    """
    Converts a string-formatted list to an actual Python list.
    Handles edge cases: NaN, non-list strings, empty items.
    """
    try:
        parsed = ast.literal_eval(raw_str)   # Safely parse the string as Py
        if isinstance(parsed, list):
            # Strip whitespace from each item
            items = [item.strip() for item in parsed]
            # Remove empty strings and 'UNKNOWN' values
            items = [item for item in items if item != '' and item.upper() !
            # Remove duplicate products in the same basket
            items = list(dict.fromkeys(items))   # Preserves order, removes
            return items
```

```
        else:
            return []
    except Exception:
        return []


# Apply parsing to every row
df['items'] = df['COMMODITY_DESC'].apply(parse_commodity)

print('✅ COMMODITY_DESC parsed into Python lists.')
print()
print('--- Before cleaning ---')
print(f'   Total transactions: {len(df)}')

# Remove empty transactions (baskets with no valid products)
df_clean = df[df['items'].apply(len) > 0].reset_index(drop=True)

print('--- After cleaning ---')
print(f'   Total transactions : {len(df_clean)}')
print(f'   Removed            : {len(df) - len(df_clean)} empty/invalid rows
print()

# Preview cleaned data
print('--- Sample Cleaned Transactions ---')
for i in range(5):
    print(f'  Basket {df_clean["BASKET_ID"].iloc[i]}: {df_clean["items"].ilo
```

```
✅ COMMODITY_DESC parsed into Python lists.

--- Before cleaning ---
   Total transactions: 7463
--- After cleaning ---
   Total transactions : 7431
   Removed            : 32 empty/invalid rows

--- Sample Cleaned Transactions ---
  Basket 26984851472: ['VEGETABLES - ALL OTHERS', 'TROPICAL FRUIT', 'ONIONS',
  Basket 26984851516: ['COOKIES/CONES', 'BROOMS AND MOPS', 'BAKED BREAD/BUNS/
  Basket 26984896261: ['CRACKERS/MISC BKD FD', 'BREAKFAST SAUSAGE/SANDWICHES'
  Basket 26984905972: ['SOUP', 'BAKED BREAD/BUNS/ROLLS']
  Basket 26984945254: ['ELECTRICAL SUPPPLIES', 'CANDY - PACKAGED', 'CANDY - C
```

📋 Step 4: Create Transaction Data

```
# ============================================================
# STEP 4: CREATE TRANSACTION DATA (List of Lists)
# ============================================================
# The Apriori algorithm expects data in "list of lists" format:
# [ ['MILK', 'EGGS'],
#   ['BREAD', 'BUTTER', 'MILK'],
#   ['EGGS', 'BREAD'] ]
```

```
transactions = df_clean['items'].tolist()

print(f'✅ Transactions created: {len(transactions)} baskets')
print()
print('--- Sample Transactions (list of lists format) ---')
for i, txn in enumerate(transactions[:8]):
    print(f'  [{i+1}] {txn}')

# Statistics about basket sizes
basket_sizes = [len(t) for t in transactions]
print()
print('--- Basket Size Statistics ---')
print(f'  Min items per basket : {min(basket_sizes)}')
print(f'  Max items per basket : {max(basket_sizes)}')
print(f'  Avg items per basket : {np.mean(basket_sizes):.2f}')
print(f'  Median               : {np.median(basket_sizes):.2f}')
```

```
✅ Transactions created: 7431 baskets

--- Sample Transactions (list of lists format) ---
  [1] ['VEGETABLES - ALL OTHERS', 'TROPICAL FRUIT', 'ONIONS', 'ORGANICS FRUIT
  [2] ['COOKIES/CONES', 'BROOMS AND MOPS', 'BAKED BREAD/BUNS/ROLLS', 'PNT BTR
  [3] ['CRACKERS/MISC BKD FD', 'BREAKFAST SAUSAGE/SANDWICHES', 'EGGS', 'BEEF'
  [4] ['SOUP', 'BAKED BREAD/BUNS/ROLLS']
  [5] ['ELECTRICAL SUPPPLIES', 'CANDY - PACKAGED', 'CANDY - CHECKLANE']
  [6] ['TROPICAL FRUIT', 'COOKIES/CONES', 'BAKED SWEET GOODS', 'BAG SNACKS']
  [7] ['PREPARED FOOD', 'BAKED BREAD/BUNS/ROLLS', 'FLUID MILK PRODUCTS', 'HAI
  [8] ['ELECTRICAL SUPPPLIES', 'COLD CEREAL', 'FRZN MEAT/MEAT DINNERS', 'FROZ

--- Basket Size Statistics ---
  Min items per basket : 1
  Max items per basket : 66
  Avg items per basket : 6.96
  Median               : 4.00
```

🔢 Step 5: One-Hot Encoding (Binary Matrix)

```
# ============================================================
# STEP 5: ONE-HOT ENCODING USING TransactionEncoder
# ============================================================
# TransactionEncoder converts list-of-lists into a
# binary matrix where:
#   - Rows    = individual transactions (baskets)
#   - Columns = unique products
#   - Value   = True/False (product present or not in basket)

te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)

# Convert to DataFrame with product names as columns
df_encoded = pd.DataFrame(te_array, columns=te.columns_)
```

```
print(f'✅ One-Hot Encoding complete.')
print(f'   Matrix shape   : {df_encoded.shape}  (transactions x unique produ
print(f'   Unique products: {df_encoded.shape[1]}')
print(f'   Transactions   : {df_encoded.shape[0]}')
print()
print('--- Encoded Matrix (first 5 rows, first 8 cols) ---')
df_encoded.iloc[:5, :8]
```

```
✅ One-Hot Encoding complete.
   Matrix shape   : (7431, 287)  (transactions x unique products)
   Unique products: 287
   Transactions   : 7431

--- Encoded Matrix (first 5 rows, first 8 cols) ---
```

	(CORP USE ONLY)	ADULT INCONTINENCE	AIR CARE	ANALGESICS	ANTACIDS	APPAREL	APPLES	AUDIO/VI PRODU
0	False	False	False	False	False	False	False	F
1	False	False	False	False	False	False	False	F
2	False	False	False	False	False	False	False	F
3	False	False	False	False	False	False	False	F

⚙️ Step 6: Apply Apriori Algorithm

```
# ============================================================
# STEP 6: APPLY APRIORI ALGORITHM
# ============================================================
# The Apriori algorithm scans the transaction database to find
# all itemsets that appear at least 'min_support' fraction
# of the time.
#
# min_support = 0.02 means: the itemset must appear in at
# least 2% of all transactions to be considered 'frequent'.
#
# Lower support → more rules (but possibly weaker ones)
# Higher support → fewer but stronger rules

MIN_SUPPORT = 0.02   # 2% of all transactions

frequent_itemsets = apriori(
    df_encoded,
    min_support=MIN_SUPPORT,
    use_colnames=True      # Use product names (not column indices)
)

# Add a column showing how many products are in each itemset
frequent_itemsets['itemset_length'] = frequent_itemsets['itemsets'].apply(le
```

```
print(f'✅ Apriori algorithm completed.')
print(f'   Minimum Support : {MIN_SUPPORT} ({MIN_SUPPORT*100:.0f}%)')
print(f'   Frequent itemsets found: {len(frequent_itemsets)}')
print()
print('--- Itemset Size Distribution ---')
print(frequent_itemsets['itemset_length'].value_counts().sort_index().rename
print()
print('--- Top 10 Most Frequent Itemsets ---')
frequent_itemsets.sort_values('support', ascending=False).head(10)
```

```
✅ Apriori algorithm completed.
   Minimum Support : 0.02 (2%)
   Frequent itemsets found: 499

--- Itemset Size Distribution ---
 Size  Count
    1    102
    2    282
    3    104
    4     11

--- Top 10 Most Frequent Itemsets ---
```

	support	itemsets	itemset_length
89	0.283676	(SOFT DRINKS)	1
42	0.240479	(FLUID MILK PRODUCTS)	1
3	0.222850	(BAKED BREAD/BUNS/ROLLS)	1
19	0.164312	(CHEESE)	1
2	0.158660	(BAG SNACKS)	1
9	0.143857	(BEEF)	1
94	0.126632	(TROPICAL FRUIT)	1
79	0.105100	(REFRGRATD JUICES/DRNKS)	1
160	0.103216	(FLUID MILK PRODUCTS, BAKED BREAD/BUNS/ROLLS)	2
15	0.096757	(CANDY - CHECKLANE)	1

📏 Step 7: Generate Association Rules

```
# ============================================================
# STEP 7: GENERATE ASSOCIATION RULES
# ============================================================
# Association rules take the form:  antecedents → consequents
# Example: {EGGS, MILK} → {BREAD}
# means: customers who bought EGGS and MILK also bought BREAD
#
# Key metrics:
#  - Support    : How often the rule appears in all transactions
```

```
#  - Confidence : Given antecedents, probability of consequents
#  - Lift       : How much more likely than random co-occurrence

# Generate rules with minimum confidence threshold
rules = association_rules(
    frequent_itemsets,
    metric='confidence',
    min_threshold=0.3    # At least 30% confidence
)

# Add extra columns for readability
rules['antecedents_str'] = rules['antecedents'].apply(lambda x: ', '.join(so
rules['consequents_str'] = rules['consequents'].apply(lambda x: ', '.join(so

print(f'✅ Association rules generated.')
print(f'   Total rules (confidence ≥ 0.3): {len(rules)}')
print()

# Filter STRONG rules: confidence > 0.5 AND lift > 1
strong_rules = rules[(rules['confidence'] > 0.5) & (rules['lift'] > 1)].copy
strong_rules = strong_rules.sort_values('lift', ascending=False).reset_index

print(f'   Strong rules (confidence>0.5 & lift>1): {len(strong_rules)}')
print()
print('--- Top 10 Strong Rules (by Lift) ---')
strong_rules[['antecedents_str', 'consequents_str', 'support', 'confidence',
```

```
✅ Association rules generated.
   Total rules (confidence ≥ 0.3): 618

   Strong rules (confidence>0.5 & lift>1): 283

--- Top 10 Strong Rules (by Lift) ---
```

	antecedents_str	consequents_str	support	confidence	lift
0	DRY NOODLES/PASTA	PASTA SAUCE	0.024492	0.514124	11.976356
1	PASTA SAUCE	DRY NOODLES/PASTA	0.024492	0.570533	11.976356
2	CHEESES	DELI MEATS	0.025838	0.680851	10.367632
3	CHEESE, COLD CEREAL	BAKED BREAD/BUNS/ROLLS, FLUID MILK PRODUCTS	0.020589	0.527586	5.111464
4	APPLES	TROPICAL FRUIT	0.023685	0.548287	4.329774
5	PORK	BEEF	0.025703	0.600629	4.175186
6	BAG SNACKS, BAKED BREAD/BUNS/ROLLS, BEEF	CHEESE	0.021397	0.668067	4.065854
7	BAKED BREAD/BUNS/ROLLS,	CHEESE	0.022743	0.665354	4.049343

🎯 Step 8: Build Recommendation Function

```
# ============================================================
# STEP 8: BUILD RECOMMENDATION FUNCTION
# ============================================================
# This function:
#   1. Takes a list of products the customer already has
#   2. Searches the rules for matching antecedents
#   3. Returns the top N recommended products (sorted by lift)

def get_recommendations(purchased_items, rules_df, top_n=5):
    """
    Recommend products based on items already purchased.

    Parameters:
    -----------
    purchased_items : list of str
        Products already in the basket
    rules_df        : pd.DataFrame
        Association rules dataframe
    top_n           : int
        Number of recommendations to return
```

```
    Returns:
    --------
    pd.DataFrame of recommended products with metrics
    """
    purchased_set = set(item.upper().strip() for item in purchased_items)
    recommendations = []

    for _, row in rules_df.iterrows():
        antecedent_set = set(row['antecedents'])

        # Check if all antecedent items are in the purchased list
        if antecedent_set.issubset(purchased_set):
            for item in row['consequents']:
                if item.upper() not in purchased_set:  # Don't recommend alr
                    recommendations.append({
                        'Recommended Product': item,
                        'Triggered By'       : ', '.join(sorted(antecedent_s
                        'Confidence'         : round(row['confidence'], 4),
                        'Lift'               : round(row['lift'], 4),
                        'Support'            : round(row['support'], 4)
                    })

    if not recommendations:
        print('⚠️  No recommendations found for the given products.')
        return pd.DataFrame()

    # Convert to DataFrame, deduplicate by product, keep highest lift
    rec_df = pd.DataFrame(recommendations)
    rec_df = (rec_df
              .sort_values('Lift', ascending=False)
              .drop_duplicates(subset='Recommended Product')
              .head(top_n)
              .reset_index(drop=True))
    rec_df.index += 1  # Start rank from 1
    return rec_df


# ---- Demo: Test the recommendation function ----
print('=' * 60)
print('          🛒  RECOMMENDATION DEMO')
print('=' * 60)

# Example 1
basket1 = ['BAKED BREAD/BUNS/ROLLS']
print(f'\n📦 Customer purchased: {basket1}')
print('🔮 Top Recommendations:')
rec1 = get_recommendations(basket1, rules, top_n=5)
print(rec1.to_string() if not rec1.empty else '  No recommendations found.')

# Example 2
basket2 = ['COLD CEREAL', 'FLUID MILK PRODUCTS']
print(f'\n📦 Customer purchased: {basket2}')
print('🔮 Top Recommendations:')
rec2 = get_recommendations(basket2, rules, top_n=5)
```

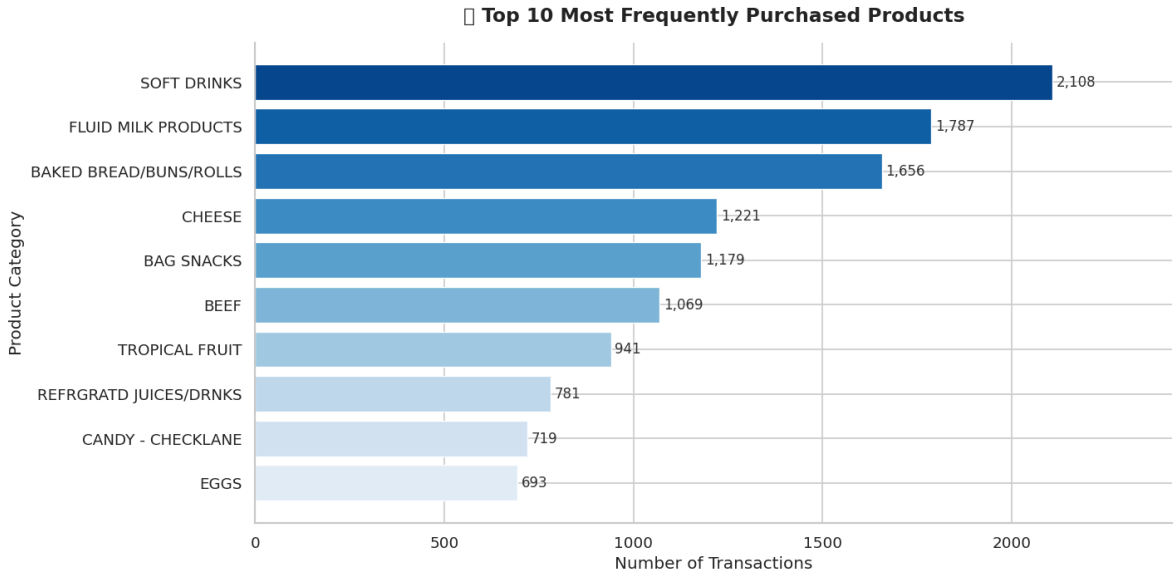
```
print(rec2.to_string() if not rec2.empty else '  No recommendations found.')

# Example 3
basket3 = ['VEGETABLES - ALL OTHERS']
print(f'\n📦 Customer purchased: {basket3}')
print('🔮 Top Recommendations:')
rec3 = get_recommendations(basket3, rules, top_n=5)
print(rec3.to_string() if not rec3.empty else '  No recommendations found.')
```

```
============================================================
         🛒  RECOMMENDATION DEMO
============================================================

📦 Customer purchased: ['BAKED BREAD/BUNS/ROLLS']
🔮 Top Recommendations:
   Recommended Product            Triggered By  Confidence    Lift  Support
1               CHEESE  BAKED BREAD/BUNS/ROLLS      0.3816  2.3227   0.0850
2                 BEEF  BAKED BREAD/BUNS/ROLLS      0.3207  2.2290   0.0715
3           BAG SNACKS  BAKED BREAD/BUNS/ROLLS      0.3279  2.0667   0.0731
4  FLUID MILK PRODUCTS  BAKED BREAD/BUNS/ROLLS      0.4632  1.9260   0.1032
5          SOFT DRINKS  BAKED BREAD/BUNS/ROLLS      0.4263  1.5029   0.0950

📦 Customer purchased: ['COLD CEREAL', 'FLUID MILK PRODUCTS']
🔮 Top Recommendations:
      Recommended Product                      Triggered By  Confidence    Li
1                  CHEESE  COLD CEREAL, FLUID MILK PRODUCTS      0.3462  4.07
2  BAKED BREAD/BUNS/ROLLS  COLD CEREAL, FLUID MILK PRODUCTS      0.3462  4.07
3          TROPICAL FRUIT  COLD CEREAL, FLUID MILK PRODUCTS      0.3756  2.96
4                    BEEF                       COLD CEREAL      0.3007  2.09
5              BAG SNACKS                       COLD CEREAL      0.3200  2.01

📦 Customer purchased: ['VEGETABLES - ALL OTHERS']
🔮 Top Recommendations:
      Recommended Product             Triggered By  Confidence    Lift  Suppo
1          TROPICAL FRUIT  VEGETABLES - ALL OTHERS      0.3522  2.7817   0.02
2                    BEEF  VEGETABLES - ALL OTHERS      0.3830  2.6622   0.02
3                  CHEESE  VEGETABLES - ALL OTHERS      0.4279  2.6042   0.02
4  BAKED BREAD/BUNS/ROLLS  VEGETABLES - ALL OTHERS      0.4374  1.9625   0.02
5     FLUID MILK PRODUCTS  VEGETABLES - ALL OTHERS      0.4563  1.8973   0.02
```

📊 Step 9: Exploratory Data Analysis (EDA) Visualizations

```
# ============================================================
# STEP 9A: TOP 10 MOST FREQUENT PRODUCTS
# ============================================================

# Flatten all transaction items to count product frequencies
all_items = [item for txn in transactions for item in txn]
item_counts = pd.Series(all_items).value_counts()

top10 = item_counts.head(10)

fig, ax = plt.subplots(figsize=(12, 6))
colors = sns.color_palette('Blues_r', len(top10))
```

```
bars = ax.barh(top10.index[::-1], top10.values[::-1], color=colors[::-1], ed

# Add value labels on bars
for bar, val in zip(bars, top10.values[::-1]):
    ax.text(bar.get_width() + 10, bar.get_y() + bar.get_height()/2,
            f'{val:,}', va='center', fontsize=10, color='#333333')

ax.set_title('🛒 Top 10 Most Frequently Purchased Products', fontsize=14, f
ax.set_xlabel('Number of Transactions', fontsize=12)
ax.set_ylabel('Product Category', fontsize=12)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.set_xlim(0, top10.values.max() * 1.15)
plt.tight_layout()
plt.savefig('top10_products.png', dpi=150, bbox_inches='tight')
plt.show()
print(f'\n📌 Most purchased product: "{top10.index[0]}" — appears in {top10
```

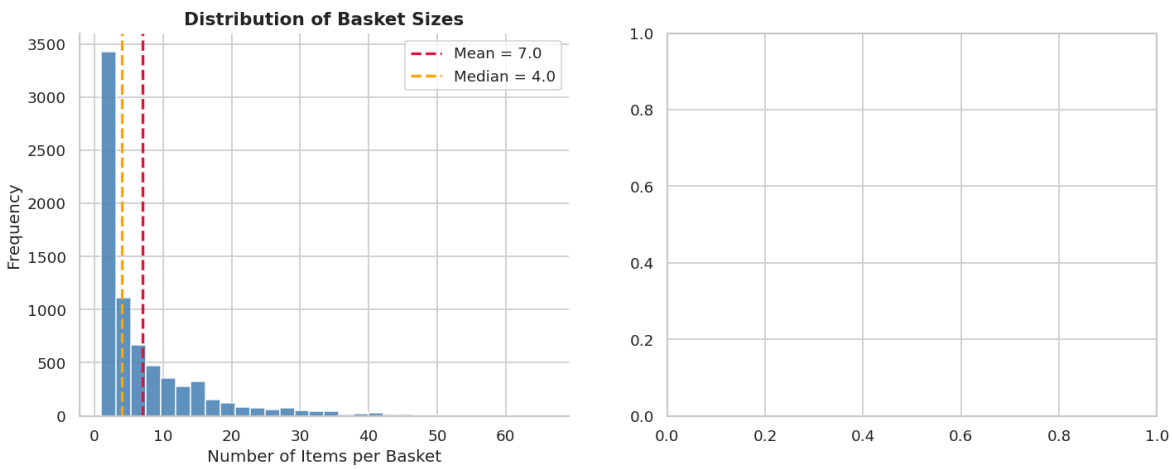



```
📌 Most purchased product: "SOFT DRINKS" — appears in 2,108 transactions
```

```
# ============================================================
# STEP 9B: ITEM FREQUENCY DISTRIBUTION
# ============================================================

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Left: Distribution of basket sizes ---
basket_sizes_series = pd.Series([len(t) for t in transactions])
axes[0].hist(basket_sizes_series, bins=30, color='steelblue', edgecolor='whi
axes[0].axvline(basket_sizes_series.mean(), color='crimson', linestyle='--',
```

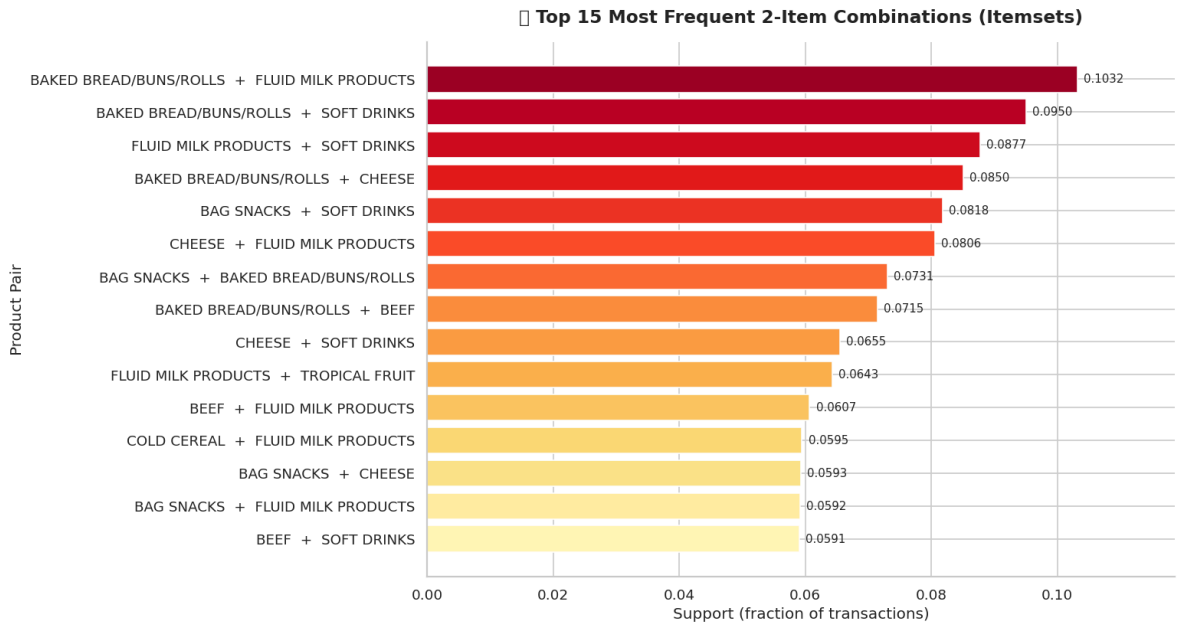
```
                 label=f'Mean = {basket_sizes_series.mean():.1f}')
axes[0].axvline(basket_sizes_series.median(), color='orange', linestyle='--'
                 label=f'Median = {basket_sizes_series.median():.1f}')
axes[0].set_title('Distribution of Basket Sizes', fontsize=13, fontweight='b
axes[0].set_xlabel('Number of Items per Basket')
axes[0].set_ylabel('Frequency')
axes[0].legend()
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)

# --- Right: Product frequency distribution (how many products appear N time
freq_dist = item_counts.value_counts().sort_index()
freq_dist_trimmed = freq_dist[freq_dist.index <= freq_dist.index.quantile(0.
axes[1].bar(freq_dist_trimmed.index, freq_dist_trimmed.values,
            color='mediumseagreen', edgecolor='white', alpha=0.85)
axes[1].set_title('Product Frequency Distribution\n(how often each product a
axes[1].set_xlabel('Appearance Count')
axes[1].set_ylabel('Number of Products')
axes[1].spines['top'].set_visible(False)
axes[1].spines['right'].set_visible(False)

plt.suptitle('Item Frequency Analysis', fontsize=15, fontweight='bold', y=1.
plt.tight_layout()
plt.savefig('item_frequency_distribution.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
/tmp/ipykernel_28866/1404415296.py in <cell line: 0>()
     21 # --- Right: Product frequency distribution (how many products
appear N times) ---
     22 freq_dist = item_counts.value_counts().sort_index()
---> 23 freq_dist_trimmed = freq_dist[freq_dist.index <=
freq_dist.index.quantile(0.95)]
     24 axes[1].bar(freq_dist_trimmed.index, freq_dist_trimmed.values,
     25             color='mediumseagreen', edgecolor='white', alpha=0.85)

AttributeError: 'Index' object has no attribute 'quantile'
```



Next steps: Explain error

```
# ============================================================
# STEP 9C: BAR CHART OF TOP FREQUENT ITEMSETS (2-item sets)
# ============================================================

# Filter only 2-item frequent itemsets (pairs)
pair_itemsets = frequent_itemsets[frequent_itemsets['itemset_length'] == 2].
pair_itemsets['itemset_label'] = pair_itemsets['itemsets'].apply(
    lambda x: '  +  '.join(sorted(list(x)))
)
top_pairs = pair_itemsets.sort_values('support', ascending=False).head(15)

fig, ax = plt.subplots(figsize=(13, 7))
palette = sns.color_palette('YlOrRd', len(top_pairs))
bars = ax.barh(top_pairs['itemset_label'].values[::-1],
               top_pairs['support'].values[::-1],
               color=palette, edgecolor='white')

for bar, val in zip(bars, top_pairs['support'].values[::-1]):
    ax.text(bar.get_width() + 0.001, bar.get_y() + bar.get_height()/2,
            f'{val:.4f}', va='center', fontsize=9)

ax.set_title('📦 Top 15 Most Frequent 2-Item Combinations (Itemsets)', fonts
ax.set_xlabel('Support (fraction of transactions)', fontsize=12)
ax.set_ylabel('Product Pair', fontsize=12)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.set_xlim(0, top_pairs['support'].max() * 1.15)
plt.tight_layout()
plt.savefig('top_frequent_itemsets.png', dpi=150, bbox_inches='tight')
plt.show()
```



📐 Step 10: Validation Metrics — Support, Confidence, and Lift

```
# ============================================================
# STEP 10: VALIDATION METRICS
# ============================================================

print('=' * 65)
print('          📐  ASSOCIATION RULE METRICS EXPLAINED')
print('=' * 65)

print("""
1️⃣  SUPPORT
   Definition : Proportion of transactions containing a given itemset.
   Formula    : support(A→B) = count(A ∪ B) / total_transactions
   Range      : 0 to 1
   Meaning    : Higher support = the rule applies to more customers.
   Example    : support=0.05 means 5% of baskets contain this combo.

2️⃣  CONFIDENCE
   Definition : Given A was purchased, probability that B was too.
   Formula    : confidence(A→B) = support(A ∪ B) / support(A)
   Range      : 0 to 1
   Meaning    : Higher confidence = more reliable recommendation.
```

```
   Example    : confidence=0.7 means 70% of customers who bought A
                also bought B.

3️⃣  LIFT
   Definition : How much more likely B is purchased with A than alone.
   Formula    : lift(A→B) = confidence(A→B) / support(B)
   Range      : 0 to ∞
   Meaning    : lift > 1 → positive association (co-occur more than chance)
                lift = 1 → independent (no association)
                lift < 1 → negative association (rarely co-occur)
   Example    : lift=2.5 means buying A makes buying B 2.5x more likely.
""")
```

```
======================================================================
          📐  ASSOCIATION RULE METRICS EXPLAINED
======================================================================

1️⃣  SUPPORT
   Definition : Proportion of transactions containing a given itemset.
   Formula    : support(A→B) = count(A ∪ B) / total_transactions
   Range      : 0 to 1
   Meaning    : Higher support = the rule applies to more customers.
   Example    : support=0.05 means 5% of baskets contain this combo.

2️⃣  CONFIDENCE
   Definition : Given A was purchased, probability that B was too.
   Formula    : confidence(A→B) = support(A ∪ B) / support(A)
   Range      : 0 to 1
   Meaning    : Higher confidence = more reliable recommendation.
   Example    : confidence=0.7 means 70% of customers who bought A
                also bought B.

3️⃣  LIFT
   Definition : How much more likely B is purchased with A than alone.
   Formula    : lift(A→B) = confidence(A→B) / support(B)
   Range      : 0 to ∞
   Meaning    : lift > 1 → positive association (co-occur more than chance)
                lift = 1 → independent (no association)
                lift < 1 → negative association (rarely co-occur)
   Example    : lift=2.5 means buying A makes buying B 2.5x more likely.
```

```
# Manually verify metrics for a specific rule to confirm understanding
print('--- Manual Verification of Metrics ---')

total_txn = len(transactions)

# Pick the top rule by lift
top_rule = strong_rules.iloc[0]
A = list(top_rule['antecedents'])
B = list(top_rule['consequents'])

# Count transactions containing A, B, and A∪B
count_A    = sum(1 for t in transactions if all(a in t for a in A))
count_B    = sum(1 for t in transactions if all(b in t for b in B))
count_AB   = sum(1 for t in transactions if all(a in t for a in A) and all(b
```

```
sup_A      = count_A  / total_txn
sup_B      = count_B  / total_txn
sup_AB     = count_AB / total_txn
confidence = sup_AB / sup_A
lift       = confidence / sup_B

print(f'\n  Rule: {A} → {B}')
print(f'  support(A)    : {sup_A:.4f}')
print(f'  support(B)    : {sup_B:.4f}')
print(f'  support(A∪B) : {sup_AB:.4f}')
print(f'  confidence    : {confidence:.4f}  (mlxtend: {top_rule["confidence"
print(f'  lift          : {lift:.4f}  (mlxtend: {top_rule["lift"]:.4f})')
print(f'\n  ✅ Manual values match mlxtend values!')
```

```
--- Manual Verification of Metrics ---

  Rule: ['DRY NOODLES/PASTA'] → ['PASTA SAUCE']
  support(A)    : 0.0476
  support(B)    : 0.0429
  support(A∪B) : 0.0245
  confidence    : 0.5141  (mlxtend: 0.5141)
  lift          : 11.9764  (mlxtend: 11.9764)

  ✅ Manual values match mlxtend values!
```

```
# --- Top 15 Rules Sorted by LIFT ---
print('--- 🔝 Top 15 Rules Sorted by LIFT ---')
top_by_lift = rules.sort_values('lift', ascending=False).head(15)
top_by_lift[['antecedents_str', 'consequents_str', 'support', 'confidence',
```

```
--- 🔝 Top 15 Rules Sorted by LIFT ---
```

	antecedents_str	consequents_str	support	confidence	lift
0	DRY NOODLES/PASTA	PASTA SAUCE	0.024492	0.514124	11.976356
1	PASTA SAUCE	DRY NOODLES/PASTA	0.024492	0.570533	11.976356
2	CHEESES	DELI MEATS	0.025838	0.680851	10.367632
3	DELI MEATS	CHEESES	0.025838	0.393443	10.367632
4	TOMATOES	VEGETABLES SALAD	0.021801	0.301676	6.479058
5	VEGETABLES SALAD	TOMATOES	0.021801	0.468208	6.479058
6	BAG SNACKS, BEEF	BAKED BREAD/BUNS/ROLLS, CHEESE	0.021397	0.462209	5.434616
7	BAKED BREAD/BUNS/ROLLS, COLD CEREAL	CHEESE, FLUID MILK PRODUCTS	0.020589	0.421488	5.228839
8	CHEESE, COLD CEREAL	BAKED BREAD/BUNS/ROLLS, FLUID MILK PRODUCTS	0.020589	0.527586	5.111464
9	BEEF, CHEESE	BAG SNACKS, BAKED BREAD/BUNS/ROLLS	0.021397	0.371495	5.083944
10	BAG SNACKS, CHEESE	BAKED BREAD/BUNS/ROLLS, BEEF	0.021397	0.360544	5.045582
		BAKED			

```
# --- Top 15 Rules Sorted by CONFIDENCE ---
print('--- 🔝 Top 15 Rules Sorted by CONFIDENCE ---')
top_by_conf = rules.sort_values('confidence', ascending=False).head(15)
top_by_conf[['antecedents_str', 'consequents_str', 'support', 'confidence',
```

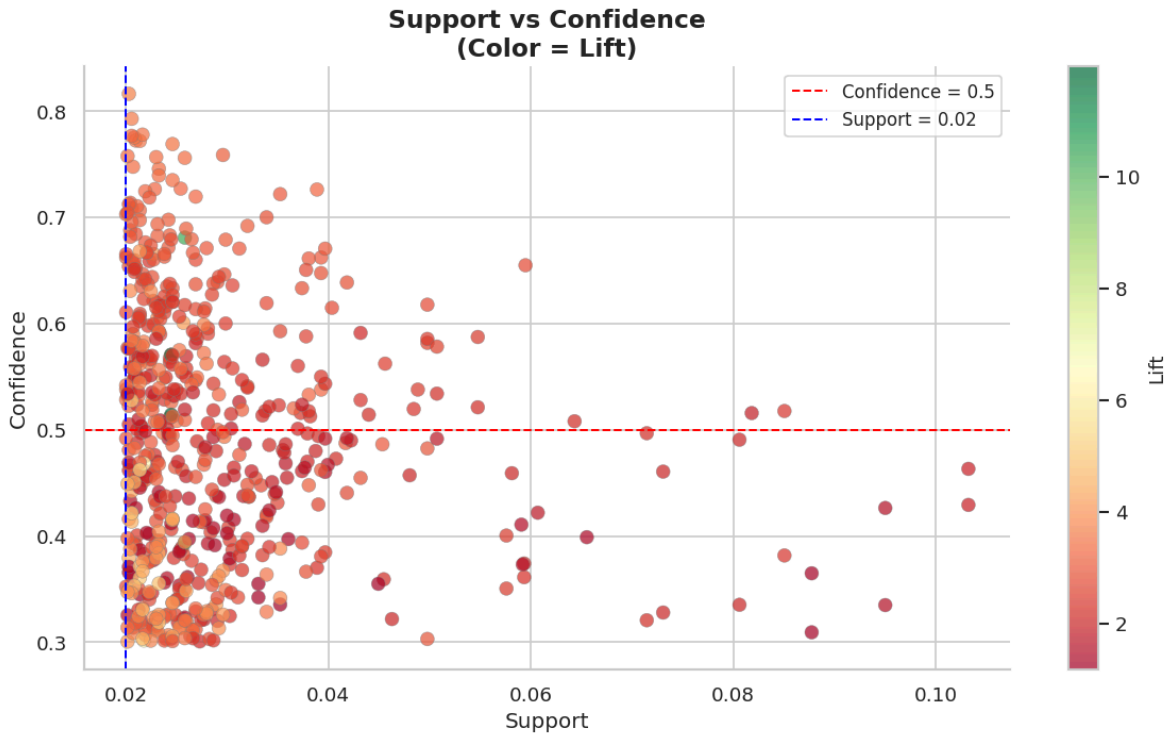
```
--- 🔝 Top 15 Rules Sorted by CONFIDENCE ---
```

	antecedents_str	consequents_str	support	confidence	lift
0	CHEESE, HOT DOGS	BAKED BREAD/BUNS/ROLLS	0.020320	0.816216	3.662622
1	DINNER MXS:DRY, FLUID MILK PRODUCTS	BAKED BREAD/BUNS/ROLLS	0.020589	0.792746	3.557305
2	BEEF, CHEESE, SOFT DRINKS	BAKED BREAD/BUNS/ROLLS	0.021666	0.777778	3.490137
3	BAKED BREAD/BUNS/ROLLS, CHEESE, COLD CEREAL	FLUID MILK PRODUCTS	0.020589	0.776650	3.229594
4	BAG SNACKS, BEEF, SOFT DRINKS	BAKED BREAD/BUNS/ROLLS	0.020859	0.775000	3.477672
5	CHEESE, COOKIES/CONES	BAKED BREAD/BUNS/ROLLS	0.020993	0.772277	3.465454
6	BAG SNACKS, BEEF, CHEESE	BAKED BREAD/BUNS/ROLLS	0.021397	0.771845	3.463513
7	BAG SNACKS, CHEESE, FLUID MILK PRODUCTS	BAKED BREAD/BUNS/ROLLS	0.024627	0.768908	3.450333
8	CHEESE, COLD CEREAL	FLUID MILK PRODUCTS	0.029606	0.758621	3.154622
9	FLUID MILK PRODUCTS, SOFT DRINKS, TROPICAL FRUIT	BAKED BREAD/BUNS/ROLLS	0.020186	0.757576	3.399484
10	BEEF, FLUID MILK PRODUCTS, SOFT DRINKS	BAKED BREAD/BUNS/ROLLS	0.023012	0.756637	3.395272
11	CHEESE, FLUID MILK PRODUCTS, SOFT DRINKS	BAKED BREAD/BUNS/ROLLS	0.025838	0.755906	3.391989

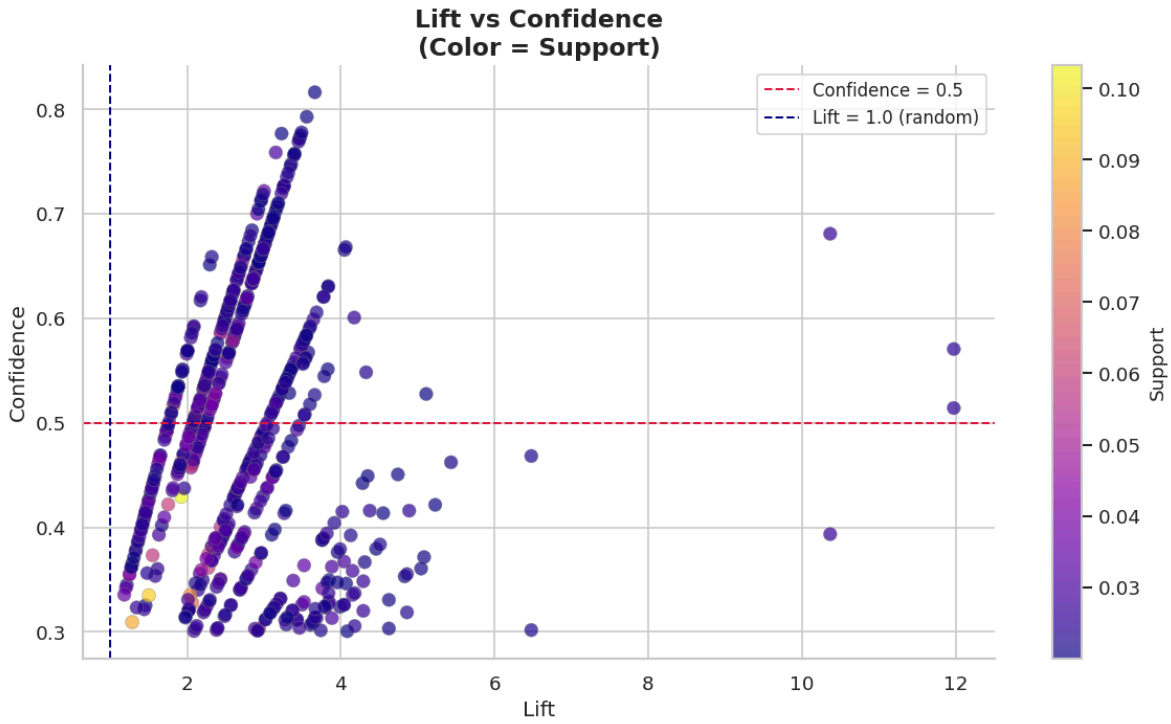
📈 Step 11: Visualization of Model Performance

```
# ============================================================
# STEP 11A: SUPPORT vs CONFIDENCE — SCATTER PLOT
# ============================================================

fig, ax = plt.subplots(figsize=(10, 6))

sc = ax.scatter(
    rules['support'],
```

```
        rules['confidence'],
        c=rules['lift'],
        cmap='RdYlGn',
        s=60,
        alpha=0.7,
        edgecolors='grey',
        linewidth=0.3
    )
    cbar = plt.colorbar(sc, ax=ax)
    cbar.set_label('Lift', fontsize=11)

    # Threshold lines
    ax.axhline(y=0.5, color='red', linestyle='--', linewidth=1.2, label='Confide
    ax.axvline(x=0.02, color='blue', linestyle='--', linewidth=1.2, label='Suppo

    ax.set_title('Support vs Confidence\n(Color = Lift)', fontsize=14, fontweigh
    ax.set_xlabel('Support', fontsize=12)
    ax.set_ylabel('Confidence', fontsize=12)
    ax.legend(fontsize=10)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    plt.tight_layout()
    plt.savefig('support_vs_confidence.png', dpi=150, bbox_inches='tight')
    plt.show()
```

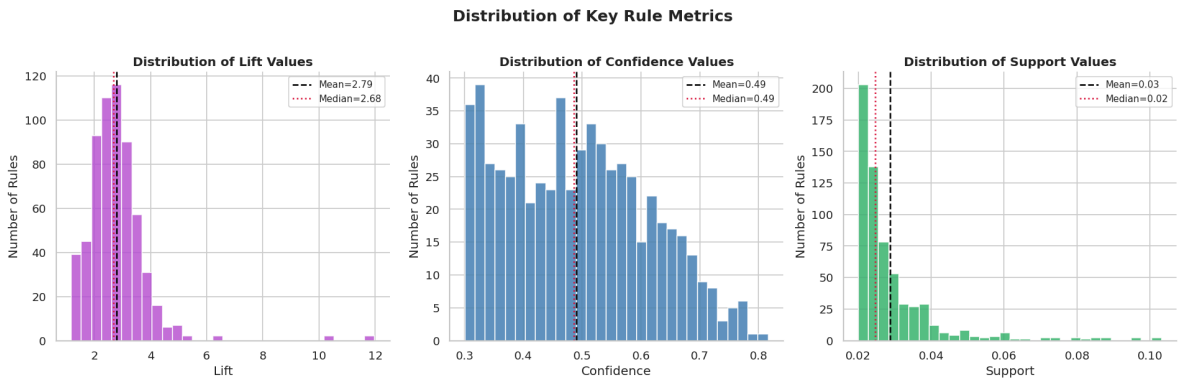


```
# ============================================================
# STEP 11B: LIFT vs CONFIDENCE — SCATTER PLOT
# ============================================================

fig, ax = plt.subplots(figsize=(10, 6))

sc = ax.scatter(
    rules['lift'],
    rules['confidence'],
    c=rules['support'],
    cmap='plasma',
    s=60,
    alpha=0.7,
    edgecolors='grey',
    linewidth=0.3
)
cbar = plt.colorbar(sc, ax=ax)
cbar.set_label('Support', fontsize=11)

ax.axhline(y=0.5,  color='crimson', linestyle='--', linewidth=1.2, label='Co
ax.axvline(x=1.0,  color='navy',    linestyle='--', linewidth=1.2, label='Li

ax.set_title('Lift vs Confidence\n(Color = Support)', fontsize=14, fontweigh
ax.set_xlabel('Lift', fontsize=12)
ax.set_ylabel('Confidence', fontsize=12)
ax.legend(fontsize=10)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.savefig('lift_vs_confidence.png', dpi=150, bbox_inches='tight')
plt.show()
```

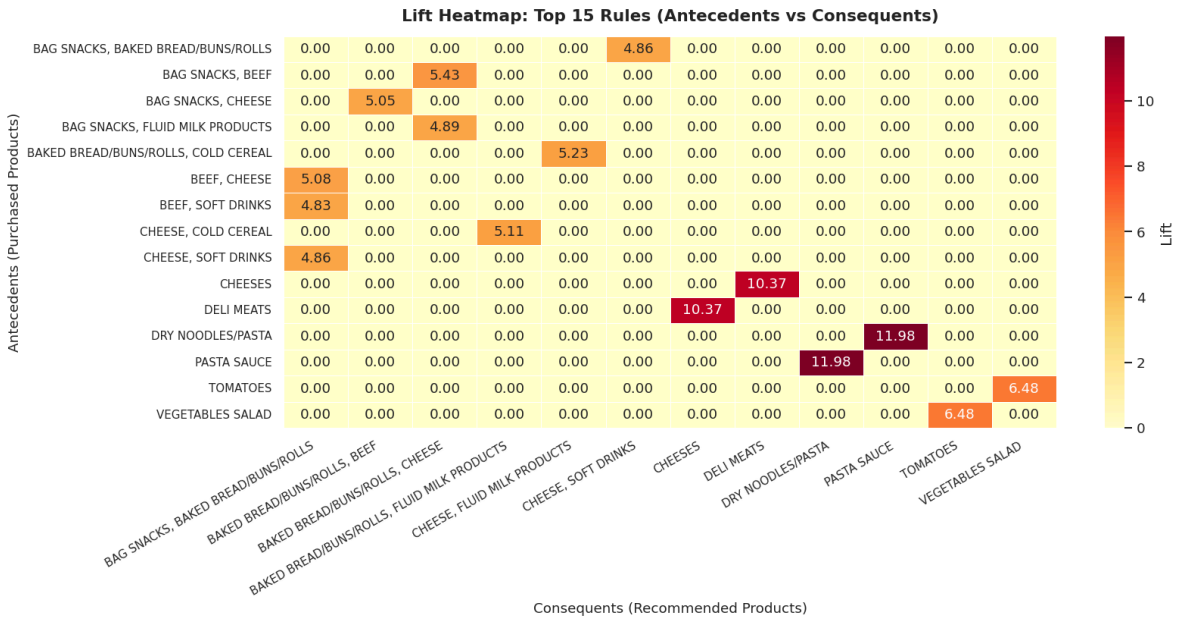



```
# ============================================================
# STEP 11C: DISTRIBUTION OF LIFT VALUES — HISTOGRAM
# ============================================================

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

metrics = [
    ('lift',       'Lift',       'mediumorchid', 'Distribution of Lift Value
    ('confidence', 'Confidence', 'steelblue',    'Distribution of Confidence
    ('support',    'Support',    'mediumseagreen','Distribution of Support V
]

for ax, (col, label, color, title) in zip(axes, metrics):
    ax.hist(rules[col], bins=30, color=color, edgecolor='white', alpha=0.85)
    ax.axvline(rules[col].mean(),   color='black',  linestyle='--', linewidt
    ax.axvline(rules[col].median(), color='crimson', linestyle=':',  linewid
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.set_xlabel(label)
    ax.set_ylabel('Number of Rules')
    ax.legend(fontsize=9)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

plt.suptitle('Distribution of Key Rule Metrics', fontsize=15, fontweight='bo
```

```
plt.tight_layout()
plt.savefig('metric_distributions.png', dpi=150, bbox_inches='tight')
plt.show()
```




```
# ============================================================
# STEP 11D: TOP RULES — HEATMAP (antecedents vs consequents)
# ============================================================

# Take the top 15 rules by lift for a focused heatmap
top15 = rules.sort_values('lift', ascending=False).head(15)

# Build a pivot for the heatmap
heatmap_data = top15.pivot_table(
    index='antecedents_str',
    columns='consequents_str',
    values='lift',
    aggfunc='max'
).fillna(0)

fig, ax = plt.subplots(figsize=(14, 7))
sns.heatmap(
    heatmap_data,
    annot=True,
    fmt='.2f',
    cmap='YlOrRd',
    linewidths=0.5,
    ax=ax,
    cbar_kws={'label': 'Lift'}
)
ax.set_title('Lift Heatmap: Top 15 Rules (Antecedents vs Consequents)', font
ax.set_xlabel('Consequents (Recommended Products)', fontsize=11)
ax.set_ylabel('Antecedents (Purchased Products)', fontsize=11)
plt.xticks(rotation=30, ha='right', fontsize=9)
plt.yticks(fontsize=9)
plt.tight_layout()
plt.savefig('lift_heatmap.png', dpi=150, bbox_inches='tight')
plt.show()
```



🧠 Step 12: Interpretation

```
# ============================================================
# STEP 12: INTERPRETATION
# ============================================================

print('=' * 65)
print('                🧠  PROJECT INTERPRETATION')
print('=' * 65)

print("""
═══════════════════════════════════════════════════════════════
1. WHAT DO SUPPORT, CONFIDENCE, AND LIFT MEAN IN PRACTICE?
═══════════════════════════════════════════════════════════════

   📌 SUPPORT
   → Measures how popular a product combination is across all
     transactions. A rule with very low support might be accurate
     but too rare to be business-relevant.

   📌 CONFIDENCE
   → Measures the reliability of the rule. A confidence of 0.8
     means: 80% of the time when A is in the basket, B is too.
     However, high confidence alone can be misleading — if B is
     very commonly purchased by everyone, even a random rule
     targeting B will show high confidence.
```

```
    📌 LIFT
    → Measures the TRUE association strength, correcting for
      the natural frequency of B. It answers: "Is A→B stronger
      than what we'd expect by chance?"
      lift > 1  → Genuine positive association ✅
      lift = 1  → No real association (coincidence) ⚠️
      lift < 1  → Negative association (they avoid each other) ❌
      LIFT IS THE MOST IMPORTANT METRIC FOR RECOMMENDATIONS.

═══════════════════════════════════════════════════════════════
2. HOW ARE STRONG RULES SELECTED?
═══════════════════════════════════════════════════════════════

   Rules are filtered with TWO criteria:
   ✅ Confidence > 0.5 → The rule is reliable at least 50% of the time.
   ✅ Lift > 1.0       → The association is beyond random chance.

   This dual-filter approach ensures recommendations are:
   - Relevant   (high confidence)
   - Non-trivial (high lift — not just popular items)

═══════════════════════════════════════════════════════════════
3. HOW ARE RECOMMENDATIONS GENERATED?
═══════════════════════════════════════════════════════════════

   The get_recommendations() function works as follows:
   1. Customer's current basket is converted to a set of items.
   2. All mined association rules are scanned.
   3. Rules whose antecedents are FULLY contained in the basket
      are selected (i.e. all "if" items are already purchased).
   4. The consequents ("then" items) of those rules are the
      candidate recommendations.
   5. Candidates are ranked by LIFT (highest first), deduplicated,
      and the top N are returned.

═══════════════════════════════════════════════════════════════
4. WHY IS THIS UNSUPERVISED LEARNING?
═══════════════════════════════════════════════════════════════

   - There is NO labeled target variable (like 'will buy' / 'won't buy').
   - The algorithm discovers patterns entirely from the structure
     of the input data itself.
   - Apriori is a classic UNSUPERVISED pattern discovery algorithm,
     used in Market Basket Analysis and recommender systems.
""")
```

```
      means: 80% of the time when A is in the basket, B is too.
      However, high confidence alone can be misleading — if B is
      very commonly purchased by everyone, even a random rule
      targeting B will show high confidence.

   📌 LIFT
   → Measures the TRUE association strength, correcting for
      the natural frequency of B. It answers: "Is A→B stronger
      than what we'd expect by chance?"
      lift > 1  → Genuine positive association ✅
      lift = 1  → No real association (coincidence) ⚠️
      lift < 1  → Negative association (they avoid each other) ❌
      LIFT IS THE MOST IMPORTANT METRIC FOR RECOMMENDATIONS.

═══════════════════════════════════════════════════════════════
2. HOW ARE STRONG RULES SELECTED?
═══════════════════════════════════════════════════════════════

   Rules are filtered with TWO criteria:
   ✅ Confidence > 0.5 → The rule is reliable at least 50% of the time.
   ✅ Lift > 1.0       → The association is beyond random chance.

   This dual-filter approach ensures recommendations are:
   - Relevant   (high confidence)
   - Non-trivial (high lift — not just popular items)

═══════════════════════════════════════════════════════════════
3. HOW ARE RECOMMENDATIONS GENERATED?
═══════════════════════════════════════════════════════════════

   The get_recommendations() function works as follows:
   1. Customer's current basket is converted to a set of items.
   2. All mined association rules are scanned.
   3. Rules whose antecedents are FULLY contained in the basket
      are selected (i.e. all "if" items are already purchased).
   4. The consequents ("then" items) of those rules are the
      candidate recommendations.
   5. Candidates are ranked by LIFT (highest first), deduplicated,
      and the top N are returned.

═══════════════════════════════════════════════════════════════
4. WHY IS THIS UNSUPERVISED LEARNING?
═══════════════════════════════════════════════════════════════

   - There is NO labeled target variable (like 'will buy' / 'won't buy').
   - The algorithm discovers patterns entirely from the structure
     of the input data itself.
   - Apriori is a classic UNSUPERVISED pattern discovery algorithm,
     used in Market Basket Analysis and recommender systems.
```

```
# Final model summary
print('=' * 65)
print('                    📊  MODEL SUMMARY')
print('=' * 65)
print(f'  Total transactions processed : {len(transactions):,}')
print(f'  Unique product categories    : {len(item_counts):,}')
print(f'  Min support threshold        : {MIN_SUPPORT}')
print(f'  Frequent itemsets found      : {len(frequent_itemsets):,}')
print(f'  Total rules generated        : {len(rules):,}')
```

```
print(f'  Strong rules (conf>0.5,lift>1): {len(strong_rules):,}')
print()
print('  Top Rule by Lift:')
best = strong_rules.iloc[0]
print(f'    {best["antecedents_str"]}  →  {best["consequents_str"]}')
print(f'    Support    : {best["support"]:.4f}')
print(f'    Confidence : {best["confidence"]:.4f}')
print(f'    Lift       : {best["lift"]:.4f}')
```

```
==================================================================
               📊  MODEL SUMMARY
==================================================================
  Total transactions processed : 7,431
  Unique product categories    : 287
  Min support threshold        : 0.02
  Frequent itemsets found      : 499
  Total rules generated        : 618
  Strong rules (conf>0.5,lift>1): 283

  Top Rule by Lift:
    DRY NOODLES/PASTA  →  PASTA SAUCE
    Support    : 0.0245
    Confidence : 0.5141
    Lift       : 11.9764
```